MRC Simulation

Repo Checkpoint 1 Submission

Repository URL: https://github.com/manoji360/MRC-Simulation

Named commit: 9f84957eea1ac87f690e11024c8dae2f295c27c8

Status: Startup-phase, Unity-first documentation repository

Milestone completion status

- CI: Pending setup; owner: Team Lead; close date: 2026-11-15

- Board: Pending setup; owner: Project Manager; close date: 2026-11-15

- Production target: No live deployment yet; repo remains planning-only; owner: Project Manager; close date: 2026-11-15

- Approval: Pending; owner: Mentor/Project Manager; close date: 2026-11-15

- Inherited-codebase audit: Not applicable; no inherited codebase yet; owner: Project Manager; close date: 2026-11-15

This PDF captures the repository state at the named commit and contains the six required living documents. The project is intentionally keeping the repo in the startup phase; it does not claim a running simulation or production deployment.

README.md

MRC Simulation

This repository is in the project-start phase. At this point, there is no production simulation implementation yet; the repository is being used to document the initial planning decisions, requirements, risks, and contribution workflow for a Unity-first Mars rover simulation. The project is not yet implemented and should not be described as a working runtime.

Project direction

The team has decided to prioritize Unity for the prototype and visual simulation work rather than Gazebo for the initial implementation. The current repo is therefore a planning and documentation scaffold, not a running rover simulation.

Quickstart

git clone https://github.com/manoji360/MRC-Simulation.git

cd MRC-Simulation

python -m venv .venv

source .venv/bin/activate

python -m pip install --upgrade pip

python -m pip install -e .[dev]

pytest -q

This does not launch a simulator yet. It validates that the repository's required planning documents and safety checks are present and consistent.

Repository structure

- `docs/requirements.md` — problem statement, requirements, constraints, and success metrics
- `docs/design.md` — system design for the early-phase Unity-oriented project
- `docs/risks.md` — risk register and external prerequisites
- `CONTRIBUTING.md` — contribution workflow and review expectations
- `AGENTS.md` and `CLAUDE.md` — AI assistant guidance
- `.github/workflows/ci.yml` — automated repository health check on pull requests and pushes to `main`
- `tests/` — lightweight verification for required project documentation and repo configuration

Current project status

The project has not started implementation work yet. The repo is intentionally minimal and reflects the beginning of the design phase, with emphasis on a Unity-friendly architecture, defined constraints, and a clean contribution process.

Planned target stack

- Unity 2022 LTS or newer for simulation, interaction, and prototype development
- Windows/macOS/Linux developer workstations depending on team tooling
- A future production pipeline may include a lightweight build/test workflow as the project matures

No Gazebo runtime, ROS 2 workspace, or production game/simulation code is expected in this checkpoint.

CONTRIBUTING.md

Contributing Guide

This repository is currently in the planning and setup phase. The project is not yet implementing a full rover system, and the work tracked here is focused on documenting the project direction, requirements, design decisions, risks, and contribution process.

Project direction

The team has chosen a Unity-first approach for the prototype and simulation work. This means the repo may eventually contain Unity scenes, scripts, and tooling, but there is currently no active simulation implementation. Gazebo and ROS are intentionally not the lead runtime choice at this checkpoint.

Getting started

Clone the repository:

git clone https://github.com/manoji360/MRC-Simulation.git

cd MRC-Simulation

Set up the local Python toolchain used for repository checks:

python -m venv .venv

source .venv/bin/activate

python -m pip install --upgrade pip

python -m pip install -e .[dev]

pytest -q

At this stage, the project is documentation-led and does not yet include a production runtime or game build.

Branching and workflow

`main` is the default integration branch. Do not commit directly to `main`.

Use a short-lived branch for each change:

git switch main

git pull --ff-only

git switch -c /-

Suggested branch prefixes:

- `feature/`
- `docs/`
- `fix/`
- `refactor/`
- `chore/`

Open a pull request to `main` when the work is ready for review and update the branch from `main` before merge.

Planning and issue tracking

Every change should be associated with a GitHub Issue. Include:

- the problem being solved or the decision being made
- acceptance criteria or review criteria
- the owner and any blockers
- how the change will be verified

Keep work small enough to review in one iteration.

Commit messages

Use Conventional Commits:

():

Examples:

docs(requirements): draft startup requirements

feat(unity): add prototype scene plan

docs(risk): update project risks

chore(ci): add repository health checks

Reference the relevant issue when applicable.

Testing and validation

This repository currently validates the planning and documentation scaffolding rather than a production simulation stack.

Run the repository checks locally:

pytest -q

The CI workflow runs the same checks on pull requests and on pushes to `main`.

Definition of done

A change is ready to merge when:

- the issue or task is completed and reviewed
- the relevant documentation reflects the current state accurately
- the test suite still passes
- no secrets or restricted materials are committed
- at least one teammate has reviewed the change

This is intentionally a lightweight definition for the early-stage repository. Product-level simulation testing will be added when the project moves beyond the planning phase.

Contribution expectations

All team members should contribute to the repository through clear, reviewable updates. The repository should stay consistent with the current project status: planning-oriented, well-documented, and easy for a new teammate to understand.

AGENTS.md

AGENTS.md

This repository is in its initial planning stage. Use CONTRIBUTING.md for the current contribution workflow and repository expectations.

Current project status

The project has not implemented a production simulation yet. The team is using this repository to capture the early requirements, design direction, risk register, and planning decisions for a Unity-first Mars rover prototype.

Tooling and runtime direction

Prefer a Unity-centric architecture for prototype development and product exploration. Gazebo and ROS are intentionally not the main technology stack at this checkpoint.

Required files

For a working repository checkpoint, keep these files accurate and consistent:

- README.md
- CONTRIBUTING.md
- docs/requirements.md
- docs/design.md
- docs/risks.md
- .github/workflows/ci.yml

Do not claim that a live simulation or deployment exists before the project has active implementation work.

docs/requirements.md

Requirements

Problem statement

The project is still in the early planning phase and does not yet have a completed rover simulation or production implementation. The immediate goal is to define a clear technical direction, scope, and repository structure for a Unity-first Mars rover prototype before building any gameplay or simulation logic.

Functional requirements

1. The repository shall capture the product goals and constraints for a Mars rover simulation.

- Acceptance criterion: the project documentation clearly explains the target use case and planned user value.

2. The system design shall prioritize a Unity-based prototype workflow.

- Acceptance criterion: the design documents identify Unity as the primary implementation tool at this stage.

3. The project shall document the high-level simulation scope and deferred work.

- Acceptance criterion: the requirements and design files identify what is in scope, what is deferred, and what is intentionally not yet implemented.

4. The repo shall include clear review and contribution guidance.

- Acceptance criterion: the contribution guide states how changes are tracked, reviewed, and merged.

5. The repository shall maintain a CI safety net for documentation and scaffolding health.

- Acceptance criterion: the automated test suite checks that mandatory project files exist and remain aligned with the repo direction.

Non-functional requirements

- The repository must support early-stage collaboration without claiming a production system that does not exist yet.

- The project must remain easy to understand for new contributors and reviewers.

- Documentation must be clear, explicit, and consistent with the project■s current status.

- The setup must remain lightweight enough to use during the early planning phase.

Constraints

- The project is not yet implementing a full simulator, game, or ROS/Gazebo runtime.

- The team prefers Unity for prototype development and visualization instead of Gazebo for the first milestone.

- Because the project is in the starting phase, the repository must describe the intended direction without overstating implementation maturity.

Success metrics

- All required planning documents are present and internally consistent.

- Contributors can clone the repo and run the repository checks without additional setup beyond the standard Python environment.

- The project status is clear to reviewers and future contributors from the README and design docs.

Non-goals

- Delivering a full rover simulation in the current checkpoint
- Implementing Gazebo-based world setup or ROS interfaces
- Claiming deployment, gameplay production readiness, or a completed simulation system

Open questions

- What exact gameplay or simulation loop should the Unity prototype demonstrate first?
- Which Mars rover behaviors should be prioritized in the first playable milestone?
- What team responsibilities and review steps are needed before a production implementation begins?

docs/design.md

Design

Context and scope

This repository is intentionally a startup-phase planning repository. It captures the project direction for a Unity-first Mars rover simulation and defines a lightweight, reviewable structure before production implementation begins.

The goal of this checkpoint is to create a clear foundation for future work. The team is not yet building a simulator runtime, and the repo should not imply that the system is more mature than it is.

Planned architecture

The early architecture is intentionally simple and conceptual. The project is expected to evolve toward a Unity-based simulation environment that supports rover behavior, camera or interaction loops, and visual validation of the prototype.

+---------------------------+

| Repository / team docs |

| README + requirements + |

| design + risk register |

+-------------+-------------+

|

v

+---------------------------+

| Planning and milestone |

| coordination |

+-------------+-------------+

|

v

+---------------------------+

| Unity-first prototype |

| scene + core gameplay |

| scripts |

+-------------+-------------+

|

v

+---------------------------+

| Future simulation and |

| validation work |

+---------------------------+

Key design choices

Unity-first direction

The project is currently planned around Unity rather than Gazebo. This decision is intentional and reflects the design goal of a visual, interactive prototype that is easier to iterate on during the startup phase.

Documentation as the initial deliverable

The repository currently functions as a planning and documentation hub. The living docs are the primary product of this checkpoint because the actual simulator has not yet started.

Minimal structure

The repo intentionally keeps a lightweight structure so the team can add implementation work later without needing to untangle an overly complex start state.

Data and interface expectations

This checkpoint does not define a production API or a finalized simulation contract. Instead, the repo establishes a clear review pipeline and planning foundation for future work.

Potential future interfaces may include:

- Unity scene objects and scripts
- rover state values and commands
- user interaction or camera controls
- gameplay loop and prototype telemetry

These are not yet final, and no product-level API should be claimed in this phase.

Deployment and runtime target

No production deployment is expected at this checkpoint. The current repository represents the setup and planning stage, and the final runtime target will be chosen when the team starts implementation. The current design direction is Unity-first rather than Gazebo-first.

Decisions intentionally deferred

- Final simulation gameplay loop
- Rover control scheme and algorithms
- Visual asset and world design
- Production deployment strategy
- Full testing strategy for runtime behavior

Open technical questions

- What is the minimum interactive rover prototype that demonstrates the required behavior?
- Which Unity scene or level will be used to validate the prototype?
- Which rover interactions and telemetry are essential in the first milestone?
- What team and tooling choices are required before a production implementation begins?

docs/risks.md

Risk Register

| Rank | Risk | Owner | Status | Mitigation |

| --- | --- | --- | --- | --- |

| 1 | The project is still in the planning stage, so scope and implementation details are not yet fully defined. | Project Manager | Open | Document the project direction early, keep requirements clear, and avoid claiming production readiness before implementation starts. |

| 2 | The team may temporarily over-focus on tool selection and delay product implementation. | Team Lead | Open | Lock the initial technology direction in the design docs and move to a limited prototype once the decision is agreed. |

| 3 | The repo could be mistaken for a working simulation if the documentation is not kept explicit about status. | AI Coordinator | Open | Keep the README, design docs, and risk register honest about current project maturity. |

| 4 | The project may drift without clear review and contribution steps during the initial phase. | Quality Owner | Open | Maintain a lightweight GitHub workflow and a simple review process from the start. |

External prerequisites

| Item | Who asked | When asked | Status |

| --- | --- | --- | --- |

| Unity prototype toolchain decision | Team / mentor | Pending | Not yet confirmed |

| Initial project scope confirmation | Project Manager | Pending | Not yet finalized |

| Review of early design direction | Student team / mentor | Pending | Not yet scheduled |